

Review of Existing CPU Pipeline Simulators

Project context: E-Learning Web Pages for Computer Science — CPU pipeline visualisation & active learning (Zain Chohan, 32001784)

1) Executive Summary

There is a healthy ecosystem of educational pipeline simulators across MIPS, DLX and RISC-V ISAs. Mature desktop tools (e.g., EduMIPS64, Ripes) offer rich control of hazards and forwarding, while several web tools (e.g., WebMIPS, WebRISC-V, U-Michigan 370 pipeline sim) prioritise accessibility with mixed depth. No single tool perfectly combines **browser-based access, cycle-accurate stage visualisation, explicit hazard tracing (RAW/WAR/WAW & control), explain-as-you-go tooltips, and assessment/quiz flow**. This leaves space for an interactive, assessment-ready web app with guided tutorials and SLEA questionnaire integration.

2) Comparative Landscape (at a glance)

Tool	ISA	Runs in Browser?	Pipeline Stages Shown	Hazards Visualised	Forwarding	Branch/Control	Notable Strengths
EduMIPS64	MIPS64	No (Java desktop)	5-stage	Data & structural	Configurable (often off by default)	Limited control strategies	Pedagogical focus; register/memory views; used in courses
DrMIPS	MIPS64	No (Java)	5-stage	Data hazards	Yes	Basic	Clear GUI; course-proven
WebMIPS	MIPS	Yes	Datapath & pipeline	Stalls/structural shown	Partial	Limited (no predictor)	Instant access; shows datapath vs control path
MIPSTView	MIPS R2000	Yes (web app)	Simple pipelined core	Basic hazards	Some	Basic	Modern UI for lectures

Tool	ISA	Runs in Browser?	Pipeline Stages Shown	Hazards Visualised	Forwarding	Branch/Control	Notable Strengths
U-Michigan 370 Pipeline Simulator	Abstract RISC	Yes	5-stage	Data & control (options)	Toggle	“Speculate & squash” option	Clean toggles to teach policies
DLX Simulators (dlxview, HASE models)	DLX	Partly (older Java/applets)	Classic 5-stage	Data hazards	Usually	Simple	Historically influential teaching tools
Ripes	RISC-V	Desktop (Qt); experimental WASM	5-stage	Data/control with waveforms	Yes	Some branch handling	Deep inspection; excellent for labs
WebRISC-V	RISC-V (RV32/64)	Yes	5-stage	Data/control; cycle-by-cycle	Yes	Educational focus	Designed for web delivery; modern

Interpretation: For our e-learning goals, **WebMIPS**, **WebRISC-V**, and **U-Michigan 370** prove that rich **browser-native** pipeline teaching works, but each lacks some combination of: fully guided feedback, detailed hazard-path explanation, and assessment integration.

3) Deeper Notes on Representative Tools

3.1 EduMIPS64 (MIPS64, desktop)

- Built to support undergraduate architecture courses; shows pipeline execution with register/memory views.
- Often configured with **no forwarding by default**, enabling clear demonstration of RAW stalls; can be toggled to compare **no/partial/full forwarding**.
- Pros: strong classroom track record, good internal state transparency.
- Cons: Java desktop dependency; not frictionless for quick web demos.

3.2 DrMIPS (MIPS64, desktop)

- Java-based, GPLv3; simulates 5-stage pipeline; includes hazards and FP support in some builds.
- Pros: intuitive interface; used for teaching.
- Cons: Requires local installation; limited web accessibility.

3.3 WebMIPS (MIPS, web)

- **Browser-based** simulation of pipelined datapath/control; shows **stall at specific stages** (IF/ID/EX/MEM/WB); instruction memory and control path are visible.
- Pros: zero-install; direct mapping to textbook datapaths.
- Cons: datapath visual is relatively **static**; minimal branch prediction options; limited assessment hooks.

3.4 U-Michigan 370 Pipeline Simulator (web)

- Web interface with **toggleable policies**: Avoid/Detect&Stall/Forwarding for data hazards; **Speculate & Squash (not-taken)** for control hazards.
- Pros: crisp UI for teaching policy trade-offs; very accessible.
- Cons: abstracted ISA/program model; not a full assembly environment.

3.5 DLX lineage (dlxview, HASE models)

- Classic teaching pipeline (DLX from Hennessy & Patterson) with visual stepping; some older Java/applets and HASE visualisations still online.
- Pros: historically canonical for pipeline concepts.
- Cons: Legacy delivery tech; not ideal for modern browsers.

3.6 Ripes (RISC-V, desktop + experimental web)

- Feature-rich visual simulator/editor; timing diagrams and stage views; used widely in RISC-V courses.
- Pros: deep visibility; strong modern UI; community maintained.
- Cons: Browser version experimental; not specifically assessment-oriented.

3.7 WebRISC-V (RISC-V, web)

- Recent **web-native** pipeline tool (RV32/64) emphasising cycle-by-cycle analysis and accessibility.
- Pros: modern, directly relevant to our delivery model; demonstrates that complex pipeline UI is practical in the browser.
- Cons: Feature completeness varies; less widespread adoption data than EduMIPS/Ripes yet.

4) What to Emulate vs. What to Improve

Emulate: - **Zero-install, web-native** delivery (WebMIPS, WebRISC-V, U-Michigan 370) for classroom spontaneity. - **Explicit policy toggles** (Forwarding on/off, stall vs. predict) to let students run A/B comparisons in one click. - **Clear state panes** (registers/memory/PC/CPI) like EduMIPS64 & Ripes.

Improve: - **Explain-as-you-go** UI: tooltips that narrate *why* a stall/bubble/forwarding path occurred, with linked mini-theory. - **Hazard overlays**: on-canvas arrows for RAW/WAR/WAW and branch-mispredict penalties. - **Assessment mode**: built-in mini-quizzes and **SLEA questionnaire** completion/consent flow. - **Exportable artefacts**: per-cycle timeline & answers for lab submission; shareable URLs for scenarios. - **Accessibility**: WCAG-conformant colour schemes; keyboard controls; screen-reader alt text for stage events.

5) Design Implications for Our Project

1. **ISA choice:** MIPS remains didactically popular; RISC-V has momentum. Either works; MIPS aligns with classic textbook exercises; RISC-V appeals to modern curricula. Consider a **pluggable ISA layer** with a minimal core first.
 2. **Visual grammar:** Use a **5-stage swimlane grid**; render **bubbles**, **forwarding arrows**, and **stage-local warnings**. Include a **hazard inspector** panel.
 3. **Pedagogy hooks:** Tutorial scripts (guided labs), **prediction of next cycle** challenges, and **confidence checks** that feed into the SLEA.
 4. **Policy toggles:** Forwarding (none/partial/full), register file write/read timing, branch strategy (not-taken, always-taken, simple predictor), structural hazards (shared MEM vs. split I/D).
 5. **Performance metrics:** CPI, stall counts by cause, branch accuracy, pipeline utilisation, speedup vs. non-pipelined.
 6. **Tech:** Web-native (HTML/CSS/JS + Canvas/SVG). Store small scenarios in URL query or localStorage; no personal data.
-

6) Gaps & Opportunities (Why this project is needed)

- **Unified web experience** that balances depth and immediate access.
 - **Narrated hazards** with interactive overlays and linked micro-explanations.
 - **Assessment-first** design (quiz mode + export + SLEA integration) absent in most current tools.
 - **Modern accessibility** and responsive layout for laptops/tablets in labs.
-

7) References (selected)

This page intentionally lists sources in your report text; see your main report's bibliography for full citations. - EduMIPS64 (papers, SourceForge page) - WebMIPS (University of Siena) - U-Michigan EECS 370 Pipeline Simulator - DLX visual simulators (dlxview; HASE models) - Ripes (Qt desktop; experimental WASM build) - WebRISC-V (web-based RV32/64 pipeline simulator)

8) Appendix: Feature Checklist for Benchmarking

- Browser-based access
- Step, run, pause; per-cycle stage view
- Data hazards: RAW/WAR/WAW detection & visual overlays
- Control hazards: branch prediction options; mispredict penalty highlight
- Structural hazards toggles (single vs split memory, RF R/W timing)
- Forwarding modes (none/partial/full) & on-canvas arrows
- Register/memory/PC panes; CPI & stall attribution
- Tutorial/quiz mode; export scenario & results
- Accessibility safeguards (contrast, keyboard, ARIA)